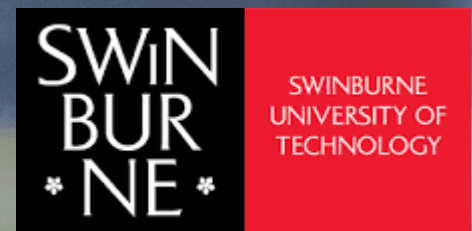




COS20028 – BIG DATA ARCHITECTURE AND APPLICATION LAB WEEK 07

Dr Pei-Wei Tsai (Lecturer, Tutor, and Unit Convenor)
ptsai@swin.edu.au, EN508d

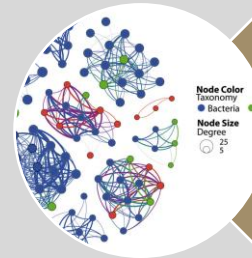




Week 7



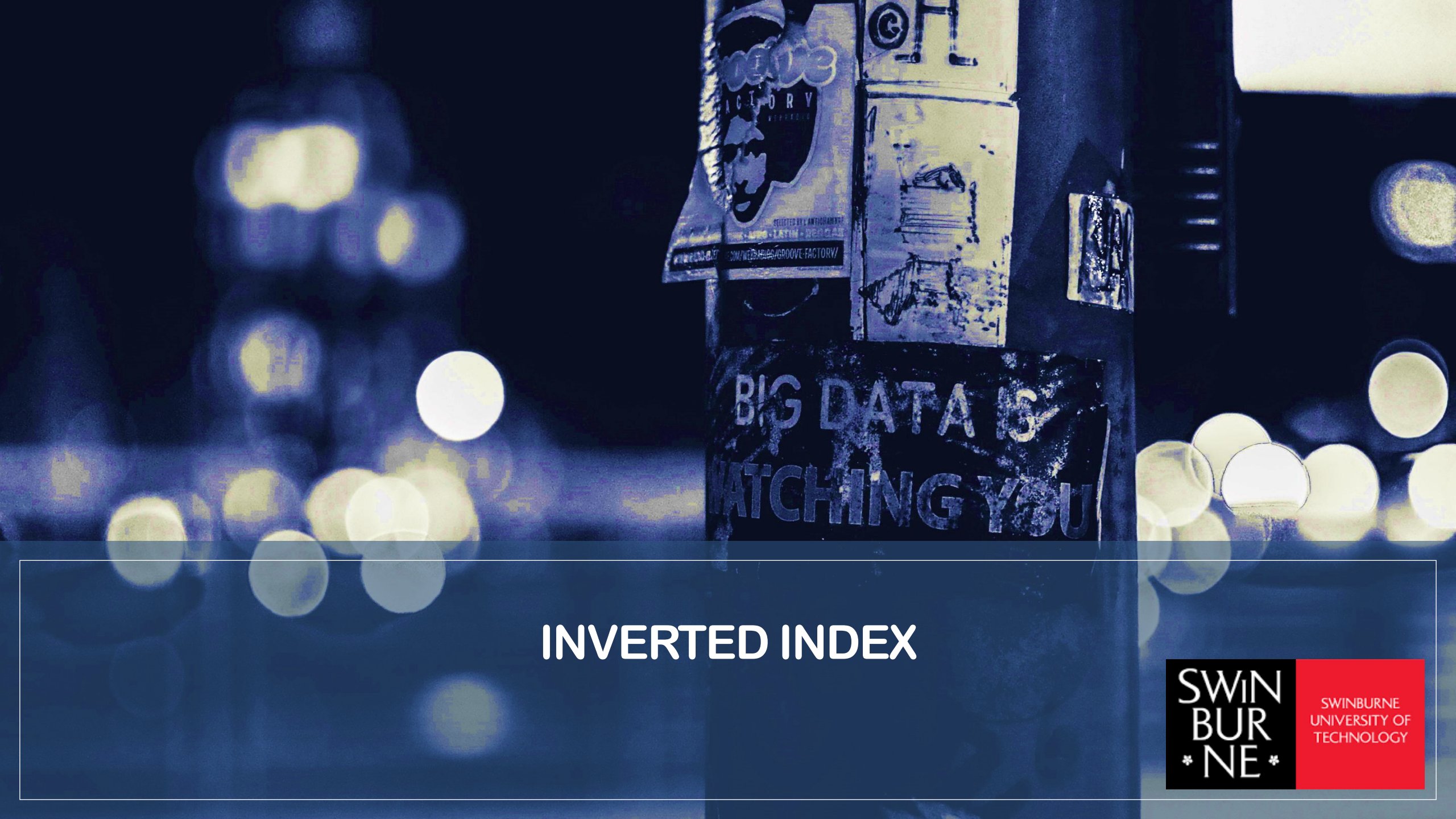
Inverted
Index



Word Co-
occurrence



Quiz 2



BIG DATA IS
WATCHING YOU

INVERTED INDEX

SWIN
BUR
NE

SWINBURNE
UNIVERSITY OF
TECHNOLOGY

CREATING AN INVERTED INDEX PROGRAM

- Writing a MapReduce job that produces an inverted index.
- You'll need to use an alternate input, provided in the file *invertedIndexInput.tgz*.
- When decompressed, this archive contains a directory of files.

Files and Directories Used in this Exercise

Eclipse project: `inverted_index`

Java files:

`IndexMapper.java` (Mapper)

`IndexReducer.java` (Reducer)

`InvertedIndex.java` (Driver)

Data files:

`~/training_materials/developer/data/invertedIndexInput.tgz`

Exercise directory: `~/workspace/inverted_index`

CREATING AN INVERTED INDEX PROGRAM

```
0      HAMLET
1
2
3      DRAMATIS PERSONAE
4
5
6      CLAUDIUS      king of Denmark. (KING CLAUDIUS:)
7
8      HAMLET  son to the late, and nephew to the present
king.
9
10     POLONIUS      lord chamberlain. (LORD POLONIUS:)
...
```

- Each file is a Shakes play formatted as the example on the left.
- Each line contains:
 - *Line_number*
 - *separator (a tab character)*
 - *Value: the line of text*
- This format can be read directly using the *KeyValueTextInputFormat* class provided in Hadoop API.
- This input format presents each line as one record to the Mapper, with the part before the tab as the key and the part after the tab as the value.

CREATING AN INVERTED INDEX PROGRAM

- Given a body of text in this form, your indexer should produce an index of all the words in the text.
- For each word, the index should have a list of all the locations, where the word appears.
- For example, for the word “honeysuckle” your output should look like this:

```
honeysuckle      2kinghenryiv@1038,midsummernightsdream@2175,...
```

The index should contain such an entry for every word in the text.


```
$ cd ~/training_materials/developer/data  
$ tar zxvf invertedIndexInput.tgz  
$ hadoop fs -put invertedIndexInput invertedIndexInput
```

INVERTED INDEX – DATA PREPARATION

```
FileSplit fileSplit = (FileSplit) context.getInputSplit();  
Path path = fileSplit.getPath();  
String fileName = path.getName();
```

RETRIEVING THE FILE NAME

Hint

- Your Mapper should take as input a key and a line of words, and emit as intermediate values each word as key, and the key as value.
- For example, the line of input from the “Hamlet” file

```
282 Have heaven and earth together
```

produces intermediate output:

```
Have      hamlet@282
heaven    hamlet@282
and       hamlet@282
earth     hamlet@282
together  hamlet@282
```

Hint

- Your Reducer simply aggregates the values presented to it for the same key, into one value.
- Use a separator such as “,” between the values.

DRIVER CODE

```
1 package stubs;
2 import org.apache.hadoop.mapreduce.Job;
3
4 import org.apache.hadoop.conf.Configured;
5 import org.apache.hadoop.conf.Configuration;
6 import org.apache.hadoop.util.Tool;
7 import org.apache.hadoop.util.ToolRunner;
8
9 import org.apache.hadoop.fs.Path;
10 import org.apache.hadoop.io.Text;
11 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
```

```
14 public class InvertedIndex extends Configured implements Tool {
15
16     public int run(String[] args) throws Exception {
17
18         Configuration conf = new Configuration();
19
20         if (args.length != 2) {
21             System.err.println("Usage: InvertedIndex <input dir> <output dir>\n");
22             return 2;
23         }
24
25         Job job = new Job(conf, "Inverted Index");
26         job.setJarByClass(InvertedIndex.class);
27         job.setJobName("Inverted Index");
28
29         // Set the Map and Reduce classes
30         job.setMapperClass(IndexMapper.class);
31         job.setReducerClass(IndexReducer.class);
32
33         // Set the output of the Map
34         job.setMapOutputKeyClass(Text.class);
35         job.setMapOutputValueClass(Text.class);
36
37         // Set the output of the Reducer
38         job.setOutputKeyClass(Text.class);
39         job.setOutputValueClass(Text.class);
40
41         // Set the directories of the input and the output
42         FileInputFormat.addInputPath(job, new Path(args[0]));
43         FileOutputFormat.setOutputPath(job, new Path(args[1]));
44
45         return (job.waitForCompletion(true) ? 0 : 1);
46     }
47
48     public static void main(String[] args) throws Exception {
49         int exitCode = ToolRunner.run(new Configuration(), new InvertedIndex(), args);
50         System.exit(exitCode);
51     }
52 }
53 }
```


MAPPER

```
1 package stubs;
2 import java.io.IOException;
3
4
5 import org.apache.hadoop.io.Text;
6 import org.apache.hadoop.io.LongWritable;
7 import org.apache.hadoop.mapreduce.Mapper;
8
9 import java.util.StringTokenizer;
10 import org.apache.hadoop.mapreduce.lib.input.FileSplit;
11
12 import org.apache.log4j.LogManager;
13 import org.apache.log4j.Logger;
14
15 public class IndexMapper extends Mapper<LongWritable, Text, Text, Text> {
16
17     private static final Logger LOGGER = LogManager.getLogger(IndexMapper.class.getName());
18
19     Text keyInfo = new Text();
20     Text valueInfo = new Text();
21     String token, lines;
22
23     @Override
24     public void map(LongWritable key, Text value, Context context) throws IOException,
25     InterruptedException {
26
27         StringTokenizer iter = new StringTokenizer(value.toString());
28         String fn = ((FileSplit) context.getInputSplit()).getPath().getName();
29
30         lines = iter.nextToken();
31
32         while(iter.hasMoreTokens()) {
33
34             token = iter.nextToken();
35             keyInfo.set(String.format("%s", token));
36             valueInfo.set(String.format("%s@%s", fn, lines));
37
38             LOGGER.info("key: " + keyInfo);
39             LOGGER.info("value: " + valueInfo);
40
41             context.write(keyInfo, valueInfo);
42         }
43     }
44 }
```

REDUCER

```
1 package stubs;
2 import java.io.IOException;
3
4 import org.apache.hadoop.io.Text;
5
6 import org.apache.hadoop.mapreduce.Reducer;
7
8 /**
9  * On input, the reducer receives a word as the key and a set
10  * of locations in the form "play name@line number" for the values.
11  * The reducer builds a readable string in the valueList variable that
12  * contains an index of all the locations of the word.
13  */
14 public class IndexReducer extends Reducer<Text, Text, Text, Text> {
15
16     Text result = new Text();
17
18     @Override
19     public void reduce(Text key, Iterable<Text> values, Context context)
20         throws IOException, InterruptedException {
21
22         StringBuffer fileList = new StringBuffer();
23         for (Text value: values) {
24             fileList.append(String.format("%s, ", value));
25         }
26         result.set(fileList.toString());
27         context.write(key, result);
28     }
29 }
```

Result

```
training@192-168-1-109:~/W7T1v1
File Edit View Search Terminal Help
&C      coriolanus@2888,coriolanus@2868,merrywivesofwindsor@2163,
&c.      merrywivesofwindsor@786,merrywivesofwindsor@1929,merrywivesofwindsor@50,
coriolanus@5559,2kinghenryiv@107,allswellthatendswell@4465,periclesprinceoftyre@
1094,tempest@2361,tempest@2291,midsummersnightsdream@941,hamlet@5260,2kinghenryv
i@1001,2kinghenryvi@111,2kinghenryvi@93,glossary@1627,loveslabourslost@47,asyoul
ikeit@1719,asyoulikeit@3780,asyoulikeit@55,asyoulikeit@3775,asyoulikeit@3770,kin
grichardiii@102,kinghenryviii@1936,kinghenryviii@1942,kinghenryviii@4033,tamingo
ftheshrew@2668,muchadoaboutnothing@44,muchadoaboutnothing@1390,3kinghenryvi@3671
,3kinghenryvi@101,titusandronicus@3406,juliuscaesar@86,juliuscaesar@993,twelfthn
ight@3999,twelfthnight@4004,twelfthnight@4012,twelfthnight@4002,twelfthnight@400
9,twelfthnight@4007,twelfthnight@3997,
&c.'      hamlet@1774,juliuscaesar@997,
&c.,      allswellthatendswell@39,kinghenryviii@4555,titusandronicus@987,titusandr
onicus@1595,
&c]      2kinghenryiv@4784,2kinghenryiv@2759,macbeth@2265,macbeth@2426,coriolanus
@5414,romeoandjuliet@2487,romeoandjuliet@4335,othello@914,1kinghenryvi@126,1king
henryvi@2078,1kinghenryvi@2972,kingrichardii@963,tempest@2747,antonyandcleopatra
@5840,2kinghenryvi@1061,2kinghenryvi@1402,hamlet@5728,hamlet@4939,hamlet@5262,ki
nglear@4943,kingjohn@2192,timonofathens@1135,timonofathens@2426,
'          cymbeline@2126,romeoandjuliet@916,rapeoflucrece@1939,rapeoflucrece@2016,
glossary@774,kingrichardiii@3758,troilusandcressida@4625,merchantofvenice@846,me
rchantofvenice@2713,hamlet@1442,
''All     loverscomplaint@213,
part-r-00000
```




BIG DATA IS
WATCHING YOU

WORD CO-OCCURRENCE

Word Co-Occurrence

- You'll write a specialisation version of Word Co-Occurrence program.
- In this case, we are only interested in pairs of words which appears **directly next** to each other.
- Complete the Mapper and the Driver.
- You can use the standard SumReducer from the WordCount project.

Files and Directories Used in this Exercise

Eclipse project: `word_co-occurrence`

Java files:

`WordCoMapper.java` (Mapper)

`SumReducer.java` (Reducer from WordCount)

`WordCo.java` (Driver)

Test directory (HDFS):

`shakespeare`

Exercise directory: `~/workspace/word_co-occurrence`

Word Co-Occurrence

- Your Mapper's intermediate output should be in the form of a Text object as the key and in IntWritable as the value.
- The key will be (*word1*, *word2*), and the value will be 1.

```

1 package stubs;
2
3 import org.apache.hadoop.fs.Path;
4 import org.apache.hadoop.io.Text;
5 import org.apache.hadoop.io.IntWritable;
6 import org.apache.hadoop.mapreduce.Job;
7 import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
8 import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
9
10 import org.apache.hadoop.conf.Configured;
11 import org.apache.hadoop.conf.Configuration;
12
13 import org.apache.hadoop.util.Tool;
14 import org.apache.hadoop.util.ToolRunner;
15
16 public class WordCo extends Configured implements Tool {
17
18     @Override
19     public int run(String[] args) throws Exception {
20
21         if (args.length != 2) {
22             System.out.printf("Usage: WordCo <input dir> <output dir>\n");
23             return -1;
24         }
25
26         Job job = new Job(getConf());
27         job.setJarByClass(WordCo.class);
28         job.setJobName("Word Co-Occurrence");
29
30         // Set the Map classes
31         job.setMapperClass(WordCoMapper.class);
32         job.setReducerClass(SumReducer.class);
33
34         // Set the output of the Map
35         job.setMapOutputKeyClass(Text.class);
36         job.setMapOutputValueClass(IntWritable.class);
37
38         // Set the output of the Reducer
39         job.setOutputKeyClass(Text.class);
40         job.setOutputValueClass(IntWritable.class);
41
42         // Set the directories of the input and the output
43         FileInputFormat.addInputPath(job, new Path(args[0]));
44         FileOutputFormat.setOutputPath(job, new Path(args[1]));
45
46         return (job.waitForCompletion(true) ? 0 : 1);
47     }
48
49     public static void main(String[] args) throws Exception {
50         int exitCode = ToolRunner.run(new Configuration(), new WordCo(), args);
51         System.exit(exitCode);
52     }
53 }

```

DRIVER CODE


```

1 package stubs;
2
3 import java.io.IOException;
4
5 import org.apache.hadoop.io.IntWritable;
6 import org.apache.hadoop.io.LongWritable;
7 import org.apache.hadoop.io.Text;
8 import org.apache.hadoop.mapreduce.Mapper;
9
10 import java.util.Arrays;
11 import java.util.Collections;
12 import java.util.List;
13
14 //import org.apache.log4j.LogManager;
15 //import org.apache.log4j.Logger;
16
17 public class WordCoMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
18
19     //private static final Logger LOGGER = LogManager.getLogger(WordCoMapper.class.getName());
20     Text keyInfo = new Text();
21     IntWritable valueInfo = new IntWritable(1);
22     String token1, token2;
23     List<String> pairs;
24
25     @Override
26     public void map(LongWritable key, Text value, Context context)
27         throws IOException, InterruptedException {
28
29         String line = value.toString();
30         boolean first = true;
31
32         for (String word : line.split("\\W+")) {
33             if (first) {
34                 token1 = word;
35                 first = false;
36             }
37             else {
38                 token2 = word;
39
40                 if (!token1.equals("") && !token2.equals("")) {
41                     pairs = Arrays.asList(token1, token2);
42                     Collections.sort(pairs, String.CASE_INSENSITIVE_ORDER);
43                     keyInfo.set(String.format("(%s,%s)", pairs.get(0), pairs.get(1)));
44                     context.write(keyInfo, valueInfo);
45                     token1 = token2;
46
47                     //LOGGER.info("pairs: " + pairs.get(0) + ", " + pairs.get(1));
48                 }
49             }
50         }
51     }
52 }

```

Mapper Code

```

1 package stubs;
2
3 import java.io.IOException;
4
5 import org.apache.hadoop.io.IntWritable;
6 import org.apache.hadoop.io.Text;
7 import org.apache.hadoop.mapreduce.Reducer;
8
9 /*
10  * To define a reduce function for your MapReduce job, subclass
11  * the Reducer class and override the reduce method.
12  * The class definition requires four parameters:
13  *   The data type of the input key (which is the output key type
14  *   from the mapper)
15  *   The data type of the input value (which is the output value
16  *   type from the mapper)
17  *   The data type of the output key
18  *   The data type of the output value
19  */
20 public class SumReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
21
22     /*
23      * The reduce method runs once for each key received from
24      * the shuffle and sort phase of the MapReduce framework.
25      * The method receives a key of type Text, a set of values of type
26      * IntWritable, and a Context object.
27      */
28     @Override
29     public void reduce(Text key, Iterable<IntWritable> values, Context context)
30         throws IOException, InterruptedException {
31         int wordCount = 0;
32
33         /*
34          * For each value in the set of values passed to us by the mapper:
35          */
36         for (IntWritable value : values) {
37
38             /*
39              * Add the value to the word count counter for this key.
40              */
41             wordCount += value.get();
42         }
43
44         /*
45          * Call the write method on the Context object to emit a key
46          * and a value from the reduce method.
47          */
48         context.write(key, new IntWritable(wordCount));
49     }
50 }

```

REDUCER CODE

```
training@192-168-1-109:~/W7T2
File Edit View Search Terminal Help
(12th,part) 1
(12th,the) 1
(1s,6d) 1
(1s,of) 1
(2d,2s) 1
(2s,capon) 1
(2s,or) 1
(6d,or) 1
(6s,8d) 1
(6s,worth) 1
(A,ABERGAVERNYY) 1
(A,ABHORSON) 1
(A,ACHILLES) 1
(A,AEGEON) 1
(A,AGRIPPA) 1
(A,AJAX) 3
(A,ALBANY) 1
(A,ALL) 2
(A,ALONSO) 1
(A,ANDREW) 1
(A,ANDRONICUS) 2
(A,ANTONIO) 3
(A,ANTONY) 1
part-r-00000
```

Result